

RAG, Agentes y Arquitecturas de IA Generativa

Retrieval-Augmented Generation, LangChain, LangGraph y patrones de diseño para sistemas GenAI

1. Retrieval-Augmented Generation (RAG)

RAG es la técnica más adoptada para dotar a los LLMs de conocimiento actualizado y específico del dominio sin necesidad de re-entrenar el modelo. Combina dos fases: recuperación (Retrieval), donde se busca información relevante en una base de conocimiento vectorial, y generación (Generation), donde el LLM produce una respuesta usando ese contexto recuperado como referencia.

1.1 Pipeline RAG Básico

- Ingesta: cargar documentos (PDF, CSV, web, APIs), dividirlos en chunks y generar sus embeddings.
- Indexación: almacenar los embeddings en una base vectorial (ChromaDB, Pinecone, FAISS, Weaviate).
- Consulta: cuando llega una pregunta, generar su embedding y buscar los chunks más similares.
- Augmentation: inyectar los chunks recuperados en el prompt como contexto.
- Generación: el LLM genera la respuesta basándose en el contexto proporcionado.

1.2 ChromaDB: Base Vectorial para Desarrollo

ChromaDB es la base de datos vectorial más popular para prototipado y proyectos de escala media. Es open source, funciona en memoria o en disco, y tiene integración nativa con LangChain. Permite persistir colecciones de documentos con sus embeddings, realizar búsquedas por similitud coseno, y filtrar por metadatos. Para producción a gran escala se prefieren soluciones como Pinecone (managed) o Weaviate (auto-hosted).

1.3 Chunking: Estrategias de Segmentación

El chunking es uno de los factores más críticos del rendimiento RAG. Estrategias: Fixed-size chunks (dividir por número de tokens, sencillo pero pierde contexto), Recursive character splitting (divide por párrafos, oraciones y palabras, el método más usado), Semantic chunking (agrupa por similitud semántica, costoso pero preciso), y Document-aware splitting (respeta la estructura del documento: secciones, tablas). El tamaño típico es 500-1000 tokens con un overlap del 10-20%.

2. Agentes de IA

Un agente de IA es un sistema que usa un LLM como motor de razonamiento para tomar decisiones sobre que acciones ejecutar. A diferencia de un pipeline estatico, el agente decide dinamicamente si necesita buscar informacion, llamar una API, ejecutar codigo o simplemente responder con su conocimiento propio.

2.1 Componentes de un Agente

- LLM (cerebro): razona, planifica y decide que hacer en cada paso.
- Herramientas (tools): funciones que el agente puede invocar (busqueda web, base de datos, calculadora, API externa).
- Memoria: historial de la conversacion actual (corto plazo) o base de conocimiento persistente (largo plazo).
- Estado: informacion sobre el progreso de la tarea actual.
- Orquestador: el framework que gestiona el ciclo razonamiento-accion (LangGraph, CrewAI, AutoGen).

2.2 Patrones de Agentes

ReAct (Reason + Act) es el patron mas extendido: el modelo razona, actua con una herramienta, observa el resultado y repite hasta tener suficiente informacion para responder. Plan-and-Execute separa la planificacion de la ejecucion: primero el modelo crea un plan de pasos, luego los ejecuta en orden. Multi-agente distribuye la tarea entre agentes especializados que colaboran (un agente investigador, uno redactor, uno revisor). Reflexion permite al agente autocriticarse y mejorar su propia respuesta.

3. LangChain y LangGraph

3.1 LangChain

LangChain es el framework mas usado para construir aplicaciones con LLMs. Proporciona abstracciones para modelos (ChatOpenAI, ChatGoogleGenerativeAI), prompts (PromptTemplate, ChatPromptTemplate), recuperadores (VectorStoreRetriever), cadenas (LCEL - LangChain Expression Language) y agentes. Soporta docenas de integraciones con bases de datos, servicios cloud y herramientas externas. LCEL (pipe operator |) permite componer cadenas de forma declarativa y eficiente.

3.2 LangGraph

LangGraph es una extension de LangChain para construir agentes como grafos de estado dirigidos. Es el framework recomendado para agentes complejos con multiples pasos, bifurcaciones condicionales y ciclos. Los conceptos clave son: StateGraph (el grafo con su estado tipado), nodes (funciones que transforman el estado), edges (conexiones entre nodos, que pueden ser condicionales) y MemorySaver (checkpoint para persistir el estado entre invocaciones).

3.3 MemorySaver y Thread IDs

MemorySaver es el mecanismo de LangGraph para implementar memoria de conversacion. Guarda el estado del grafo despues de cada nodo (checkpoint). Cada conversacion se identifica con un thread_id unico, lo que permite multiples conversaciones paralelas y completamente aisladas. En produccion, MemorySaver puede sustituirse por SqliteSaver o PostgresSaver para persistencia real entre reinicios del servidor.

4. Evaluacion de Sistemas RAG

Evaluar la calidad de un sistema RAG es fundamental. Las metricas principales son: Faithfulness (la respuesta esta basada en el contexto recuperado, sin inventar), Answer Relevancy (la respuesta responde a la pregunta formulada), Context Precision (los documentos recuperados son relevantes para la pregunta) y Context Recall (se recupero toda la informacion necesaria). Frameworks como RAGAS automatizan estas evaluaciones usando el propio LLM como juez.

5. Herramientas del Ecosistema GenAI

Google AI Studio: IDE web gratuito para experimentar con modelos Gemini. Permite probar prompts, generar claves API y acceder a Gemini Flash Lite sin coste.

Groq: Plataforma de inferencia ultrarapida para modelos abiertos (LLaMA, Mistral). Ofrece tier gratuito generoso. Latencia muy baja gracias a chips LPU propios.

LangSmith: Plataforma de observabilidad de LangChain. Permite trazar, evaluar y monitorizar aplicaciones LangChain/LangGraph en produccion.

Streamlit: Framework Python para crear interfaces web de IA en minutos. Sin necesidad de conocimientos de frontend. Ideal para demos y prototipos.

Ollama: Herramienta para ejecutar modelos abiertos localmente (LLaMA, Mistral, Gemma). Interfaz similar a Docker para gestionar modelos.

HuggingFace: El repositorio central de modelos abiertos, datasets y demos (Spaces). Tambien ofrece Inference API para ejecutar modelos sin GPU propia.

6. Consideraciones de Produccion

Desplegar un sistema RAG/agente en produccion requiere atender varios aspectos criticos: gestion segura de API keys (variables de entorno, nunca en el codigo), limitacion de tasa (rate limiting) para controlar costes, logging y trazabilidad de todas las interacciones, guardrails para detectar y filtrar outputs inapropiados, fallbacks cuando el modelo no tiene informacion suficiente, y monitorizacion continua de la calidad de las respuestas. El uso de .env con python-dotenv es la practica estandar para el desarrollo local.